Image Processing and Enhancement

Blurring an Image

In [3]:
```
import cv2
from matplotlib import pyplot as plt

image_path = "image.jfif"
image = cv2.imread(image_path)
resized_image = cv2.resize(image, (1900, 800))
resized_image_rgb = cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB)
plt.imshow(resized_image_rgb)
plt.title('Original Image')
plt.axis('off')
plt.show()
```

In [4]:
```
Gaussian = cv2.GaussianBlur(resized_image, (15, 15), 0)
Gaussian_rgb = cv2.cvtColor(Gaussian, cv2.COLOR_BGR2RGB)
plt.imshow(Gaussian_rgb)
plt.title('Gaussian Blurred Image')
plt.axis('off')
plt.show()
```

Gaussian Blurred Image

In [5]:
```
median = cv2.medianBlur(resized_image, 11)
median_rgb = cv2.cvtColor(median, cv2.COLOR_BGR2RGB)

plt.imshow(median_rgb)
plt.title('Median Blurred Image')
plt.axis('off')
plt.show()
```

Median Blurred Image

In [6]:
```
bilateral = cv2.bilateralFilter(resized_image, 15, 150, 150)
bilateral_rgb = cv2.cvtColor(bilateral, cv2.COLOR_BGR2RGB)

plt.imshow(bilateral_rgb)
plt.title('Bilateral Blurred Image')
plt.axis('off')
plt.show()
```

Bilateral Blurred Image

Grayscaling of Images

In [7]:
```
import cv2

image = cv2.imread('image.jfif')

if image is None:
    print("Image not found. Check filename.")
else:
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    cv2.imshow('Grayscale', gray_image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

In [8]:
```
import cv2

img = cv2.imread('image.jfif', 0)

cv2.imshow('Grayscale Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [9]:
```
import cv2

img_weighted = cv2.imread('image.jfif')
rows, cols = img_weighted.shape[:2]

for i in range(rows):
    for j in range(cols):
        gray = 0.2989 * img_weighted[i, j][2] + 0.5870 * img_weighted[i, j][1] + 0.
        img_weighted[i, j] = [gray, gray, gray]

cv2.imshow('Grayscale Image (Weighted)', img_weighted)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [10]:
```
import cv2

img = cv2.imread('image.jfif')
rows, cols = img.shape[:2]

for i in range(rows):
    for j in range(cols):
        gray = (img[i, j, 0] + img[i, j, 1] + img[i, j, 2]) / 3
        img[i, j] = [gray, gray, gray]

cv2.imshow('Grayscale Image (Average)', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

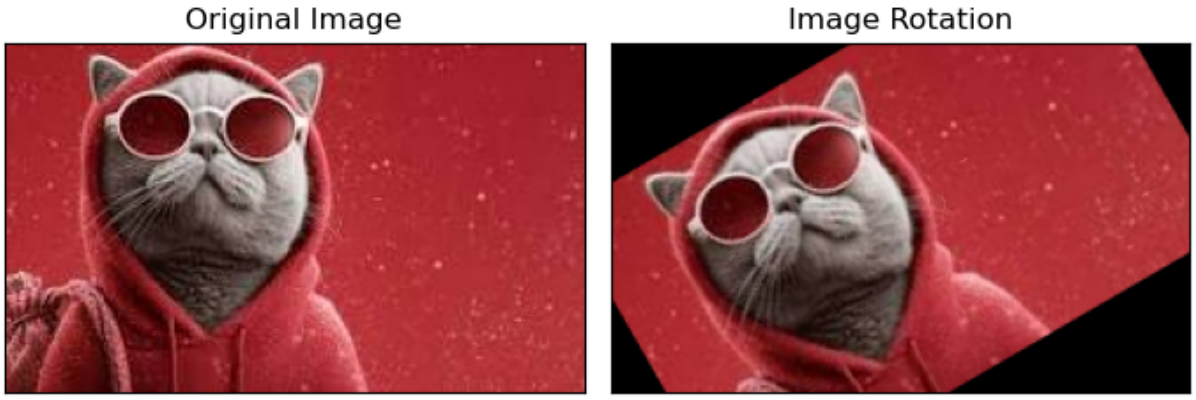
```
C:\Users\HP\AppData\Local\Temp\ipykernel_3288\3191414705.py:8: RuntimeWarning: overf
low encountered in scalar add
  gray = (img[i, j, 0] + img[i, j, 1] + img[i, j, 2]) / 3
```

Scaling, Rotating, Shifting and Edge Detection

In [11]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
scale_factor_1 = 3.0
scale_factor_2 = 1/3.0
height, width = image_rgb.shape[:2]
new_height = int(height * scale_factor_1)
new_width = int(width * scale_factor_1)

zoomed_image = cv2.resize(src =image_rgb,
                          dsize=(new_width, new_height),
                          interpolation=cv2.INTER_CUBIC)

new_height1 = int(height * scale_factor_2)
new_width1 = int(width * scale_factor_2)
scaled_image = cv2.resize(src= image_rgb,
                          dsize =(new_width1, new_height1),
                          interpolation=cv2.INTER_AREA)

fig, axs = plt.subplots(1, 3, figsize=(10, 4))
axs[0].imshow(image_rgb)
axs[0].set_title('Original Image Shape:'+str(image_rgb.shape))
axs[1].imshow(zoomed_image)
axs[1].set_title('Zoomed Image Shape:'+str(zoomed_image.shape))
axs[2].imshow(scaled_image)
axs[2].set_title('Scaled Image Shape:'+str(scaled_image.shape))

for ax in axs:
    ax.set_xticks([])
    ax.set_yticks([])
```

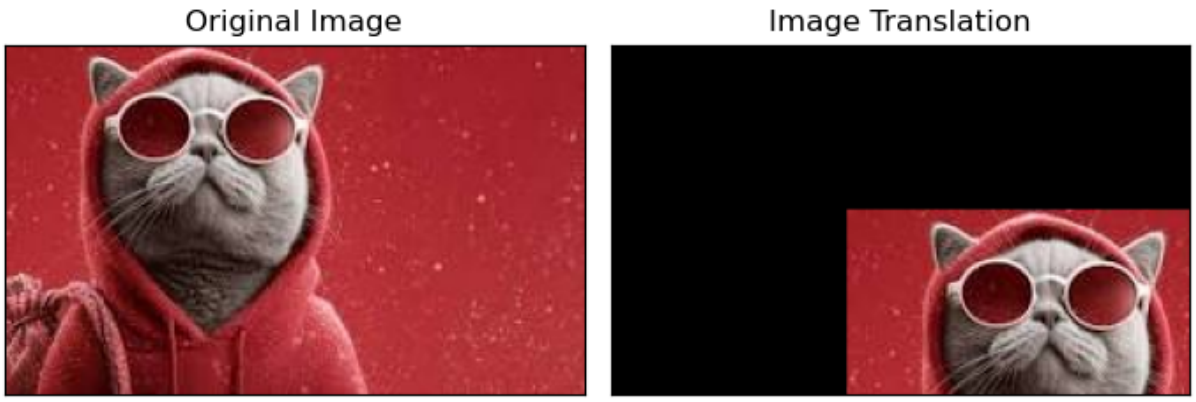
```
plt.tight_layout()
plt.show()
```

In [12]:
```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
center = (image_rgb.shape[1] // 2, image_rgb.shape[0] // 2)
angle = 30
scale = 1
rotation_matrix = cv2.getRotationMatrix2D(center, angle, scale)
rotated_image = cv2.warpAffine(image_rgb, rotation_matrix, (img.shape[1], img.shape

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb)
axs[0].set_title('Original Image')
axs[1].imshow(rotated_image)
axs[1].set_title('Image Rotation')
for ax in axs:
    ax.set_xticks([])
    ax.set_yticks([])

plt.tight_layout()
plt.show()
```



In [13]:
```
import cv2
import matplotlib.pyplot as plt
import numpy as np

img = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
width, height = image_rgb.shape[1], image_rgb.shape[0]

tx, ty = 100, 70
```

```
translation_matrix = np.array([[1, 0, tx], [0, 1, ty]], dtype=np.float32)
translated_image = cv2.warpAffine(image_rgb, translation_matrix, (width, height))

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb), axs[0].set_title('Original Image')
axs[1].imshow(translated_image), axs[1].set_title('Image Translation')

for ax in axs:
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```



In [14]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
width, height = image_rgb.shape[1], image_rgb.shape[0]

shearX, shearY = -0.15, 0
transformation_matrix = np.array([[1, shearX, 0], [0, 1, shearY]], dtype=np.float32
sheared_image = cv2.warpAffine(image_rgb, transformation_matrix, (width, height))

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb), axs[0].set_title('Original Image')
axs[1].imshow(sheared_image), axs[1].set_title('Sheared Image')

for ax in axs:
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```

In [15]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
b, g, r = cv2.split(image_rgb)

b_normalized = cv2.normalize(b.astype('float'), None, 0, 1, cv2.NORM_MINMAX)
g_normalized = cv2.normalize(g.astype('float'), None, 0, 1, cv2.NORM_MINMAX)
r_normalized = cv2.normalize(r.astype('float'), None, 0, 1, cv2.NORM_MINMAX)

normalized_image = cv2.merge((b_normalized, g_normalized, r_normalized))
print(normalized_image[:, :, 0])

plt.imshow(normalized_image)
plt.xticks([]),
plt.yticks([]),
plt.title('Normalized Image')
plt.show()
```

```
[[0.64081633 0.63673469 0.62857143 ... 0.64897959 0.66530612 0.65306122]
 [0.63265306 0.62857143 0.6244898  ... 0.60816327 0.62040816 0.61632653]
 [0.6244898  0.62040816 0.61632653 ... 0.59183673 0.60408163 0.60816327]
 ...
 [0.44897959 0.4122449  0.3755102  ... 0.64489796 0.68979592 0.7755102 ]
 [0.4244898  0.39591837 0.37142857 ... 0.67755102 0.68979592 0.73877551]
 [0.40816327 0.39183673 0.3755102  ... 0.71020408 0.68979592 0.68979592]]
```

Normalized Image

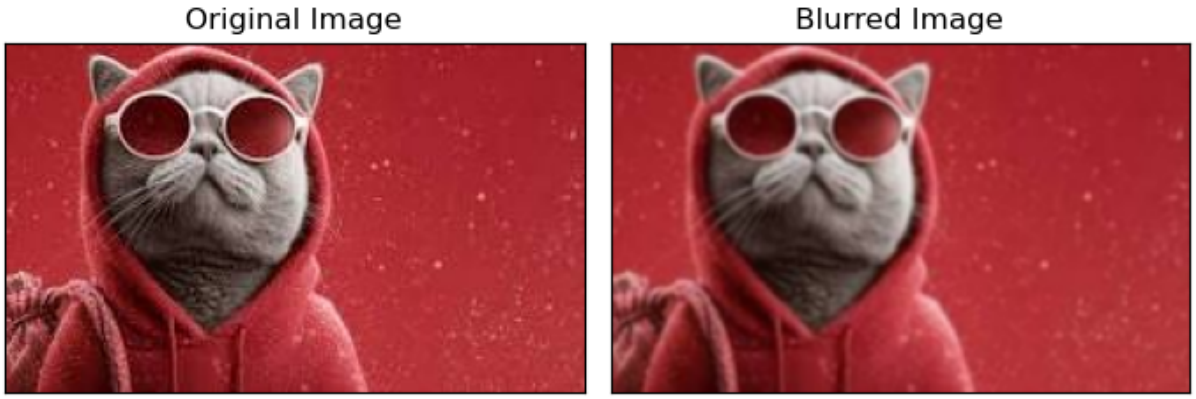
In [16]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
edges = cv2.Canny(image_rgb, 100, 700)

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb), axs[0].set_title('Original Image')
axs[1].imshow(edges), axs[1].set_title('Image Edges')

for ax in axs:
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```

Original Image

Image Edges

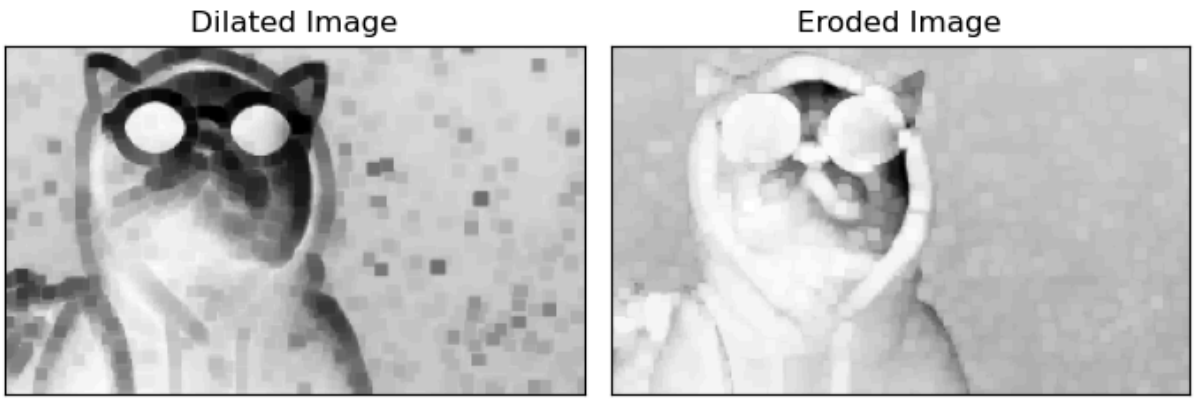
In [17]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

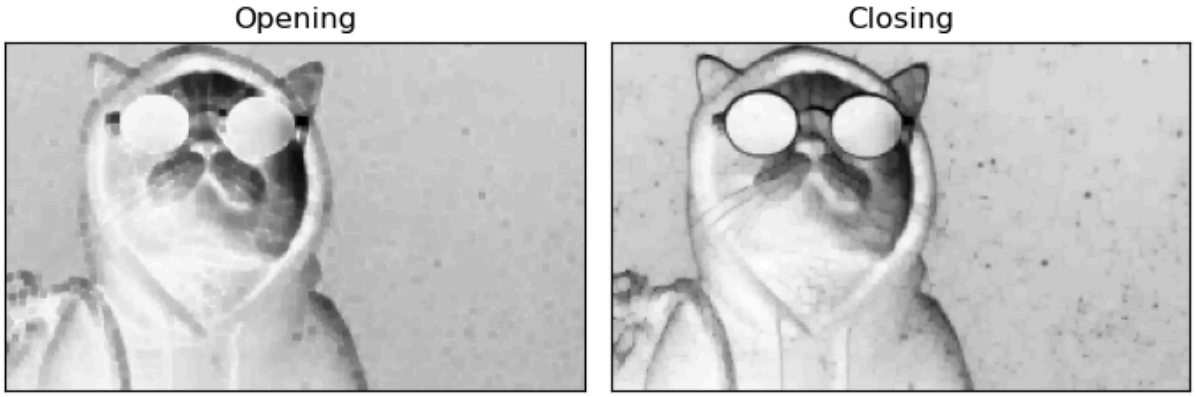
```
image = cv2.imread('image.jfif')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
blurred = cv2.GaussianBlur(image, (3, 3), 0)
blurred_rgb = cv2.cvtColor(blurred, cv2.COLOR_BGR2RGB)

fig, axs = plt.subplots(1, 2, figsize=(7, 4))
axs[0].imshow(image_rgb), axs[0].set_title('Original Image')
axs[1].imshow(blurred_rgb), axs[1].set_title('Blurred Image')

for ax in axs:
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```



In [18]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jfif')
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
kernel = np.ones((3, 3), np.uint8)

dilated = cv2.dilate(image_gray, kernel, iterations=2)
eroded = cv2.erode(image_gray, kernel, iterations=2)
opening = cv2.morphologyEx(image_gray, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(image_gray, cv2.MORPH_CLOSE, kernel)

fig, axs = plt.subplots(2, 2, figsize=(7, 7))
axs[0, 0].imshow(dilated, cmap='Greys'), axs[0, 0].set_title('Dilated Image')
axs[0, 1].imshow(eroded, cmap='Greys'), axs[0, 1].set_title('Eroded Image')
axs[1, 0].imshow(opening, cmap='Greys'), axs[1, 0].set_title('Opening')
axs[1, 1].imshow(closing, cmap='Greys'), axs[1, 1].set_title('Closing')

for ax in axs.flatten():
    ax.set_xticks([]), ax.set_yticks([])

plt.tight_layout()
plt.show()
```





Intensity Transformation Operations on Images

In [19]:
```
import cv2
import numpy as np

# Open the image.
img = cv2.imread('image.jfif')

# Apply log transform.
c = 255/(np.log(1 + np.max(img)))
log_transformed = c * np.log(1 + img)

# Specify the data type.
log_transformed = np.array(log_transformed, dtype = np.uint8)

# Save the output.
cv2.imwrite('log_transformed.jpg', log_transformed)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_3288\3021610328.py:9: RuntimeWarning: divid
e by zero encountered in log
  log_transformed = c * np.log(1 + img)
C:\Users\HP\AppData\Local\Temp\ipykernel_3288\3021610328.py:12: RuntimeWarning: inva
lid value encountered in cast
  log_transformed = np.array(log_transformed, dtype = np.uint8)
```

Out[19]: True

In [20]:
```python
import cv2
import numpy as np

# Open the image.
img = cv2.imread('image.jfif')

# Trying 4 gamma values.
for gamma in [0.1, 0.5, 1.2, 2.2]:

    # Apply gamma correction.
    gamma_corrected = np.array(255*(img / 255) ** gamma, dtype = 'uint8')

    # Save edited images.
    cv2.imwrite('gamma_transformed'+str(gamma)+'.jpg', gamma_corrected)
```

In [22]:
```python
# Contrast =  (I_max - I_min)/(I_max + I_min)
```

In [23]:
```python
import cv2
import numpy as np

# Function to map each intensity level to output intensity level.
def pixelVal(pix, r1, s1, r2, s2):
    if (0 <= pix and pix <= r1):
        return (s1 / r1)*pix
    elif (r1 < pix and pix <= r2):
        return ((s2 - s1)/(r2 - r1)) * (pix - r1) + s1
    else:
        return ((255 - s2)/(255 - r2)) * (pix - r2) + s2

# Open the image.
img = cv2.imread('image.jfif')

# Define parameters.
r1 = 70
s1 = 0
r2 = 140
s2 = 255

# Vectorize the function to apply it to each value in the Numpy array.
pixelVal_vec = np.vectorize(pixelVal)

# Apply contrast stretching.
contrast_stretched = pixelVal_vec(img, r1, s1, r2, s2)

# Save edited image.
cv2.imwrite('contrast_stretch.jpg', contrast_stretched)
```

Out[23]:
```
True
```

Image Translation

In [ ]:
```python
import cv2
import numpy as np
```

```
from google.colab.patches import cv2_imshow
image = cv2.imread('image.jfif')

height, width = image.shape[:2]
quarter_height, quarter_width = height / 4, width / 4

T = np.float32([[1, 0, quarter_width], [0, 1, quarter_height]])

img_translation = cv2.warpAffine(image, T, (width, height))

cv2_imshow(image)
cv2_imshow(img_translation)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[19], line 13
      9 T = np.float32([[1, 0, quarter_width], [0, 1, quarter_height]])
     11 img_translation = cv2.warpAffine(image, T, (width, height))
---> 13 cv2_imshow(image)
     14 cv2_imshow(img_translation)

NameError: name 'cv2_imshow' is not defined
```

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('/content/retriver.webp')
rows, cols, _ = img.shape

M_left = np.float32([[1, 0, -50], [0, 1, 0]])
M_right = np.float32([[1, 0, 50], [0, 1, 0]])
M_top = np.float32([[1, 0, 0], [0, 1, 50]])
M_bottom = np.float32([[1, 0, 0], [0, 1, -50]])

img_left = cv2.warpAffine(img, M_left, (cols, rows))
img_right = cv2.warpAffine(img, M_right, (cols, rows))
img_top = cv2.warpAffine(img, M_top, (cols, rows))
img_bottom = cv2.warpAffine(img, M_bottom, (cols, rows))

plt.subplot(221), plt.imshow(img_left), plt.title('Left')
plt.subplot(222), plt.imshow(img_right), plt.title('Right')
plt.subplot(223), plt.imshow(img_top), plt.title('Top')
plt.subplot(224), plt.imshow(img_bottom), plt.title('Bottom')
plt.show()
```

Image Pyramid

In [ ]:
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

image = cv2.imread('/content/pyramid sample.webp')

downsampled_image = cv2.pyrDown(image)
```

```
cv2_imshow(image)
cv2_imshow(downsampled_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

image = cv2.imread('/content/pyramid sample.webp')

upsampled_image = cv2.pyrUp(image)

cv2_imshow(image)
cv2_imshow(upsampled_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

image = cv2.imread('/content/pyramid sample.webp')

pyramid = [image]

for i in range(3):
    image = cv2.pyrDown(image)
    pyramid.append(image)

for i in range(len(pyramid)-1, -1, -1):
    print(f"Pyramid Level {i}")
    cv2_imshow(pyramid[i])
    cv2.waitKey(0)

cv2.destroyAllWindows()
```

Histograms Equalization

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [ ]:
```
img = cv2.imread('/content/koala sample.jpeg', 0)
equ = cv2.equalizeHist(img)
res = np.hstack((img, equ))
```

In [ ]:
```
plt.figure(figsize=(10, 5))
plt.imshow(res, cmap='gray')
plt.title("Original vs Equalized Image")
plt.axis('off')
plt.show()
```

Convert an image from one color space to another

In [ ]:
```
import cv2
src = cv2.imread(r'logo.png')  # Read the image

# Convert to Grayscale
gray_image = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)

# Display
cv2.imshow("Grayscale Image", gray_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import cv2
src = cv2.imread(r'logo.png')

# Convert to HSV
hsv_image = cv2.cvtColor(src, cv2.COLOR_BGR2HSV)

cv2.imshow("HSV Image", hsv_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import cv2
import matplotlib.pyplot as plt
src = cv2.imread(r'logo.png')

# Convert from BGR to RGB
rgb_image = cv2.cvtColor(src, cv2.COLOR_BGR2RGB)

# Display with Matplotlib
plt.imshow(rgb_image)
plt.title("RGB Image for Matplotlib")
plt.axis('off')
plt.show()
```

Visualizing image in different color spaces

In [ ]:
```
# Python program to read image as RGB

# Importing cv2 and matplotlib module
import cv2
import matplotlib.pyplot as plt

# reads image as RGB
img = cv2.imread('g4g.png')

# shows the image
plt.imshow(img)
```

In [ ]:
```
# Python program to read image as GrayScale
```

```
# Importing cv2 module
import cv2

# Reads image as gray scale
img = cv2.imread('g4g.png', 0)

# We can alternatively convert
# image by using cv2color
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Shows the image
cv2.imshow('image', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
# Python program to read image
# as YCrCb color space

# Import cv2 module
import cv2

# Reads the image
img = cv2.imread('g4g.png')

# Convert to YCrCb color space
img = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)

# Shows the image
cv2.imshow('image', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
# Python program to read image
# as HSV color space

# Importing cv2 module
import cv2

# Reads the image
img = cv2.imread('g4g.png')

# Converts to HSV color space
img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Shows the image
cv2.imshow('image', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
# Python program to read image
# as LAB color space
```

```
# Importing cv2 module
import cv2

# Reads the image
img = cv2.imread('g4g.png')

# Converts to LAB color space
img = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

# Shows the image
cv2.imshow('image', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
# Python program to read image
# as EdgeMap

# Importing cv2 module
import cv2

# Reads the image
img = cv2.imread('g4g.png')


laplacian = cv2.Laplacian(img, cv2.CV_64F)
cv2.imshow('EdgeMap', laplacian)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
# Python program to visualize
# Heat map of image

# Importing matplotlib and cv2
import matplotlib.pyplot as plt
import cv2

# reads the image
img = cv2.imread('g4g.png')

# plot heat map image
plt.imshow(img, cmap ='hot')
```

In [ ]:
```
# Python program to visualize
# Spectral map of image

# Importing matplotlib and cv2
import matplotlib.pyplot as plt
import cv2

img = cv2.imread('g4g.png')
plt.imshow(img, cmap ='nipy_spectral')
```

Create Border around Images

In [ ]:
```
import cv2

image = cv2.imread("logo.png")
image = cv2.copyMakeBorder( image, 10, 10, 10, 10, cv2.BORDER_CONSTANT, value=(0, 0

cv2.imshow("Bordered Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import cv2

image = cv2.imread("logo.png")

border_reflect = cv2.copyMakeBorder(image, 50, 50, 50, 50, cv2.BORDER_REFLECT)
border_reflect_101 = cv2.copyMakeBorder(image, 50, 50, 50, 50, cv2.BORDER_REFLECT_1
border_replicate = cv2.copyMakeBorder(image, 50, 50, 50, 50, cv2.BORDER_REPLICATE)

cv2.imshow("Border Reflect", border_reflect)
cv2.imshow("Border Reflect 101", border_reflect_101)
cv2.imshow("Border Replicate", border_replicate)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import cv2

image = cv2.imread("logo.png")
bordered_image = cv2.copyMakeBorder(image, 10, 10, 10, 10, cv2.BORDER_CONSTANT, val

cv2.imshow("Red Border Image", bordered_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Image Segmentation and Thresholding

Simple Thresholding

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('input.png')
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

In [ ]:
```
def show_image(img, title):
    plt.imshow(img, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.show()
```

In [ ]:
```
show_image(gray_image, 'Original Grayscale Image')
```

In [ ]:
```
_, thresh_binary = cv2.threshold(gray_image, 120, 255, cv2.THRESH_BINARY)
show_image(thresh_binary, 'Binary Threshold ')
```

In [ ]:
```
_, thresh_binary_inv = cv2.threshold(
    gray_image, 120, 255, cv2.THRESH_BINARY_INV)
show_image(thresh_binary_inv, 'Binary Threshold Inverted ')
```

In [ ]:
```
_, thresh_trunc = cv2.threshold(gray_image, 120, 255, cv2.THRESH_TRUNC)
show_image(thresh_trunc, 'Truncated Threshold')
```

In [ ]:
```
_, thresh_tozero = cv2.threshold(gray_image, 120, 255, cv2.THRESH_TOZERO)
show_image(thresh_tozero, 'Set to 0 ')
```

In [ ]:
```
_, thresh_tozero_inv = cv2.threshold(
    gray_image, 120, 255, cv2.THRESH_TOZERO_INV)
show_image(thresh_tozero_inv, 'Set to 0 Inverted')
```

Adaptive Thresholding

In [ ]:
```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('input1.jpg')
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

In [ ]:
```
def show_image(img, title):
    plt.imshow(img, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.show()
```

In [ ]:
```
show_image(gray_image, "Original Grayscale Image")
```

In [ ]:
```
thresh_mean = cv2.adaptiveThreshold(
    gray_image, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    199, 5
)
show_image(thresh_mean, "Adaptive Mean Thresholding")
```

In [ ]:
```
thresh_gauss = cv2.adaptiveThreshold(
    gray_image, 255,
```

```
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    199, 5
)
show_image(thresh_gauss, "Adaptive Gaussian Thresholding")
```

Morphological Operations & Filtering

Erosion and Dilation of images

In [ ]:
```
img = cv2.imread('input.png', 0)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.show()
kernel = np.ones((5, 5), np.uint8)
```

In [ ]:
```
img_erosion = cv2.erode(img, kernel, iterations=1)

plt.imshow(img_erosion, cmap='gray')
plt.title("After Erosion")
plt.axis('off')
plt.show()
```

In [ ]:
```
img_dilation = cv2.dilate(img, kernel, iterations=1)

plt.imshow(img_dilation, cmap='gray')
plt.title("After Dilation")
plt.axis('off')
plt.show()
```

Bilateral Filtering

In [ ]:
```
import cv2

# Read the image.
img = cv2.imread('taj.jpg')

# Apply bilateral filter with d = 15,
# sigmaColor = sigmaSpace = 75.
bilateral = cv2.bilateralFilter(img, 15, 75, 75)

# Save the output.
cv2.imwrite('taj_bilateral.jpg', bilateral)
```

Denoising of colored images

In [ ]:
```
# Importing required libraries
import numpy as np
import cv2
```

```
from matplotlib import pyplot as plt

# Reading the noisy image from file
img = cv2.imread('bear.png')

# Applying denoising filter
dst = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 15)

# Displaying original and denoised images
plt.subplot(121), plt.imshow(img), plt.title('Original Image')
plt.subplot(122), plt.imshow(dst), plt.title('Denoised Image')
plt.show()
```

Filter Color with OpenCV

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)  # Start webcam

while True:
    _, frame = cap.read()

    # Convert BGR to HSV
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # Define range for blue color in HSV
    lower_blue = np.array([60, 35, 140])
    upper_blue = np.array([180, 255, 255])

    # Create mask
    mask = cv2.inRange(hsv, lower_blue, upper_blue)

    # Filter the blue region
    result = cv2.bitwise_and(frame, frame, mask=mask)

    # Show frames
    cv2.imshow('Original Frame', frame)
    cv2.imshow('Blue Mask', mask)
    cv2.imshow('Blue Filtered Result', result)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break  # Exit the loop when 'q' is pressed

cap.release()
cv2.destroyAllWindows()
print("Exiting...")  # Confirm exit
```

Advanced Image Manipulation & Background Subtraction

Image Inpainting using OpenCV

In [ ]:
```
import cv2
import numpy as np

# reading the damaged image
damaged_img = cv2.imread(filename=r"cat_damaged.png")

# get the shape of the image
height, width = damaged_img.shape[0], damaged_img.shape[1]

# Converting all pixels greater than zero to black while black becomes white
for i in range(height):
    for j in range(width):
        if damaged_img[i, j].sum() > 0:
            damaged_img[i, j] = 0
        else:
            damaged_img[i, j] = [255, 255, 255]

# saving the mask
mask = damaged_img
cv2.imwrite('mask.jpg', mask)

# displaying mask
cv2.imshow("damaged image mask", mask)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

In [ ]:
```
import numpy as np
import cv2

# Open the image.
img = cv2.imread('cat_damaged.png')

# Load the mask.
mask = cv2.imread('cat_mask.png', 0)

# Inpaint.
dst = cv2.inpaint(img, mask, 3, cv2.INPAINT_NS)

# Write the output.
cv2.imwrite('cat_inpainted.png', dst)
```

Image Registration

In [ ]:
```
import cv2
import numpy as np

# Open the image files.
img1_color = cv2.imread("align.jpg")  # Image to be aligned.
img2_color = cv2.imread("ref.jpg")    # Reference image.

# Convert to grayscale.
```

```
img1 = cv2.cvtColor(img1_color, cv2.COLOR_BGR2GRAY)
img2 = cv2.cvtColor(img2_color, cv2.COLOR_BGR2GRAY)
height, width = img2.shape

# Create ORB detector with 5000 features.
orb_detector = cv2.ORB_create(5000)

# Find keypoints and descriptors.
# The first arg is the image, second arg is the mask
#  (which is not required in this case).
kp1, d1 = orb_detector.detectAndCompute(img1, None)
kp2, d2 = orb_detector.detectAndCompute(img2, None)

# Match features between the two images.
# We create a Brute Force matcher with
# Hamming distance as measurement mode.
matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck = True)

# Match the two sets of descriptors.
matches = matcher.match(d1, d2)

# Sort matches on the basis of their Hamming distance.
matches.sort(key = lambda x: x.distance)

# Take the top 90 % matches forward.
matches = matches[:int(len(matches)*0.9)]
no_of_matches = len(matches)

# Define empty matrices of shape no_of_matches * 2.
p1 = np.zeros((no_of_matches, 2))
p2 = np.zeros((no_of_matches, 2))

for i in range(len(matches)):
  p1[i, :] = kp1[matches[i].queryIdx].pt
  p2[i, :] = kp2[matches[i].trainIdx].pt

# Find the homography matrix.
homography, mask = cv2.findHomography(p1, p2, cv2.RANSAC)

# Use this matrix to transform the
# colored image wrt the reference image.
transformed_img = cv2.warpPerspective(img1_color,
                    homography, (width, height))

# Save the output.
cv2.imwrite('output.jpg', transformed_img)
```

Background subtraction

In [ ]:

```
import numpy as np
import cv2

# Load video file
cap = cv2.VideoCapture('/home/sourabh/Downloads/people-walking.mp4')
```

```
# Create background subtractor (MOG2 handles shadows well)
fgbg = cv2.createBackgroundSubtractorMOG2()

while True:
    ret, frame = cap.read()
    if not ret:
        break  # Stop if video ends

    # Apply background subtraction
    fgmask = fgbg.apply(frame)

    # Show original and foreground mask side by side
    cv2.imshow('Original Frame', frame)
    cv2.imshow('Foreground Mask', fgmask)

    # Press 'Esc' to exit
    if cv2.waitKey(30) & 0xFF == 27:
        break

# Release resources
cap.release()
cv2.destroyAllWindows()
```

Background Subtraction in an Image using Concept of Running Average

In [ ]:

```
import cv2
import numpy as np

# Capture video from webcam
cap = cv2.VideoCapture(0)

# Read the first frame and convert to float
_, img = cap.read()
averageValue1 = np.float32(img)

while True:
    # Capture next frame
    _, img = cap.read()

    # Update background model
    cv2.accumulateWeighted(img, averageValue1, 0.02)

    # Convert back to 8-bit for display
    resultingFrames1 = cv2.convertScaleAbs(averageValue1)

    # Show both original and background model
    cv2.imshow('Original Frame', img)
    cv2.imshow('Background (Running Average)', resultingFrames1)

    # Exit on Esc key
    if cv2.waitKey(30) & 0xFF == 27:
        break

# Cleanup
```

```
cap.release()
cv2.destroyAllWindows()
```

Foreground Extraction in an Image using Grabcut Algorithm

In [ ]:

```
# Python program to illustrate foreground extraction using GrabCut algorithm

# organize imports
import numpy as np
import cv2
from matplotlib import pyplot as plt

# path to input image specified and
# image is loaded with imread command
image = cv2.imread('image.jpg')

# create a simple mask image similar
# to the loaded image, with the
# shape and return type
mask = np.zeros(image.shape[:2], np.uint8)

# specify the background and foreground model
# using numpy the array is constructed of 1 row
# and 65 columns, and all array elements are 0
# Data type for the array is np.float64 (default)
backgroundModel = np.zeros((1, 65), np.float64)
foregroundModel = np.zeros((1, 65), np.float64)

# define the Region of Interest (ROI)
# as the coordinates of the rectangle
# where the values are entered as
# (startingPoint_x, startingPoint_y, width, height)
# these coordinates are according to the input image
# it may vary for different images
rectangle = (20, 100, 150, 150)

# apply the grabcut algorithm with appropriate
# values as parameters, number of iterations = 3
# cv2.GC_INIT_WITH_RECT is used because
# of the rectangle mode is used
cv2.grabCut(image, mask, rectangle,
            backgroundModel, foregroundModel,
            3, cv2.GC_INIT_WITH_RECT)

# In the new mask image, pixels will
# be marked with four flags
# four flags denote the background / foreground
# mask is changed, all the 0 and 2 pixels
# are converted to the background
# mask is changed, all the 1 and 3 pixels
# are now the part of the foreground
# the return type is also mentioned,
# this gives us the final mask
mask2 = np.where((mask == 2)|(mask == 0), 0, 1).astype('uint8')
```

```
# The final mask is multiplied with
# the input image to give the segmented image.
image_segmented = image * mask2[:, :, np.newaxis]

# output segmented image with colorbar
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off')

# Display the segmented image
plt.subplot(1, 2, 2)
plt.title('Segmented Image')
plt.imshow(cv2.cvtColor(image_segmented, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.show()
```

Feature Detection and Description

Line detection using Houghline method

In [ ]:

```
# Python program to illustrate HoughLine
# method for line detection
import cv2
import numpy as np

# Reading the required image in
# which operations are to be done.
# Make sure that the image is in the same
# directory in which this python program is
img = cv2.imread('image.jpg')

# Convert the img to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply edge detection method on the image
edges = cv2.Canny(gray, 50, 150, apertureSize=3)

# This returns an array of r and theta values
lines = cv2.HoughLines(edges, 1, np.pi/180, 200)

# The below for loop runs till r and theta values
# are in the range of the 2d array
for r_theta in lines:
    arr = np.array(r_theta[0], dtype=np.float64)
    r, theta = arr
    # Stores the value of cos(theta) in a
    a = np.cos(theta)

    # Stores the value of sin(theta) in b
    b = np.sin(theta)
```

```
    # x0 stores the value rcos(theta)
    x0 = a*r

    # y0 stores the value rsin(theta)
    y0 = b*r

    # x1 stores the rounded off value of (rcos(theta)-1000sin(theta))
    x1 = int(x0 + 1000*(-b))

    # y1 stores the rounded off value of (rsin(theta)+1000cos(theta))
    y1 = int(y0 + 1000*(a))

    # x2 stores the rounded off value of (rcos(theta)+1000sin(theta))
    x2 = int(x0 - 1000*(-b))

    # y2 stores the rounded off value of (rsin(theta)-1000cos(theta))
    y2 = int(y0 - 1000*(a))

    # cv2.line draws a line in img from the point(x1,y1) to (x2,y2).
    # (0,0,255) denotes the colour of the line to be
    # drawn. In this case, it is red.
    cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 2)

# All the changes made in the input image are finally
# written on a new image houghlines.jpg
cv2.imwrite('linesDetected.jpg', img)
```

In [ ]:

```
import cv2
import numpy as np

# Read image
image = cv2.imread('path/to/image.png')

# Convert image to grayscale
gray = cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)

# Use canny edge detection
edges = cv2.Canny(gray,50,150,apertureSize=3)

# Apply HoughLinesP method to
# to directly obtain line end points
lines_list =[]
lines = cv2.HoughLinesP(
            edges, # Input edge image
            1, # Distance resolution in pixels
            np.pi/180, # Angle resolution in radians
            threshold=100, # Min number of votes for valid line
            minLineLength=5, # Min allowed length of line
            maxLineGap=10 # Max allowed gap between line for joining them
            )

# Iterate over points
for points in lines:
      # Extracted points nested in the list
    x1,y1,x2,y2=points[0]
    # Draw the lines joing the points
```

```
    # On the original image
    cv2.line(image,(x1,y1),(x2,y2),(0,255,0),2)
    # Maintain a simples lookup list for points
    lines_list.append([(x1,y1),(x2,y2)])

# Save the result image
cv2.imwrite('detectedLines.png',image)
```

Circle Detection

In [ ]:
```
import cv2
import numpy as np

# Read image
img = cv2.imread("eyes.jpg")
output = img.copy()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Reduce noise
gray = cv2.medianBlur(gray, 5)

# Detect circles
circles = cv2.HoughCircles(
    gray,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=100,
    param1=100,
    param2=40,
    minRadius=30,
    maxRadius=60
)

# Draw only the first detected circle
if circles is not None:
    circles = np.uint16(np.around(circles))
    x, y, r = circles[0][0]
    cv2.circle(output, (x, y), r, (0, 255, 0), 2)  # Circle outline
    cv2.circle(output, (x, y), 2, (0, 0, 255), 3)  # Center point

# Show result
cv2.imshow('Detected Circle', output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Detect corner of an image

In [ ]:
```
import numpy as np
import cv2
from matplotlib import pyplot as plt
```

In [ ]:
```
img = cv2.imread('corner1.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

In [ ]:
```
corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=27,
    qualityLevel=0.01,
    minDistance=10,
    blockSize=3,
    useHarrisDetector=False,
    k=0.04
)
```

In [ ]:
```
corners = np.intp(corners)  # Convert to integer coords
for corner in corners:
    x, y = corner.ravel()
    cv2.circle(img, (x, y), radius=3, color=(0, 255, 0), thickness=-1)

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Corners Detected')
plt.axis('off')
plt.show()
```

Corner Detection with Shi-Tomasi method

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('/content/inputchess.png')

gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

corners = cv2.goodFeaturesToTrack(gray_img, 100, 0.01, 10)
corners = np.int32(corners)

for i in corners:
    x, y = i.ravel()
    cv2.circle(img, (x, y), 3, (0, 0, 255), -1)

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('Shi-Tomasi Corner Detection')
plt.axis('off')
plt.show()
```

Corner detection with Harris Corner Detection

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image_path = "/content/sample_image.jpg"
image = cv2.imread(image_path)
```

```
operatedImage = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
operatedImage = np.float32(operatedImage)


dest = cv2.cornerHarris(operatedImage, 17, 21, 0.01)
dest = cv2.dilate(dest, None)

image[dest > 0.01 * dest.max()] = [0, 0, 255]
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(image_rgb)
plt.title('Harris Corner Detection')
plt.axis('off')
plt.show()
```

Find Circles and Ellipses in an Image

In [ ]:
```
Circularity = 4*pi*Area/(perimeter)^2.
```

In [ ]:
```
import cv2
import numpy as np

# Load image
image = cv2.imread('C://gfg//images//blobs.jpg', 0)

# Set our filtering parameters
# Initialize parameter setting using cv2.SimpleBlobDetector
params = cv2.SimpleBlobDetector_Params()

# Set Area filtering parameters
params.filterByArea = True
params.minArea = 100

# Set Circularity filtering parameters
params.filterByCircularity = True
params.minCircularity = 0.9

# Set Convexity filtering parameters
params.filterByConvexity = True
params.minConvexity = 0.2

# Set inertia filtering parameters
params.filterByInertia = True
params.minInertiaRatio = 0.01

# Create a detector with the parameters
detector = cv2.SimpleBlobDetector_create(params)

# Detect blobs
keypoints = detector.detect(image)

# Draw blobs on our image as red circles
blank = np.zeros((1, 1))
blobs = cv2.drawKeypoints(image, keypoints, blank, (0, 0, 255),
```

```
                            cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

number_of_blobs = len(keypoints)
text = "Number of Circular Blobs: " + str(len(keypoints))
cv2.putText(blobs, text, (20, 550),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 100, 255), 2)

# Show blobs
cv2.imshow("Filtering Circular Blobs Only", blobs)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Document field detection

In [ ]:

```
# importing libraries
import numpy as np
import imutils
import cv2

field_threshold = { "prev_policy_no" : 0.7,
                    "address"        : 0.6,
                  }

# Function to Generate bounding
# boxes around detected fields
def getBoxed(img, img_gray, template, field_name = "policy_no"):

    w, h = template.shape[::-1]

    # Apply template matching
    res = cv2.matchTemplate(img_gray, template,
                            cv2.TM_CCOEFF_NORMED)

    hits = np.where(res >= field_threshold[field_name])

    # Draw a rectangle around the matched region.
    for pt in zip(*hits[::-1]):
        cv2.rectangle(img, pt, (pt[0] + w, pt[1] + h),
                                    (0, 255, 255), 2)

        y = pt[1] - 10 if pt[1] - 10 > 10 else pt[1] + h + 20

        cv2.putText(img, field_name, (pt[0], y),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 1)

    return img


# Driver Function
if __name__ == '__main__':

    # Read the original document image
    img = cv2.imread('doc.png')

    # 3-d to 2-d conversion
```

```
    img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Field templates
    template_add = cv2.imread('doc_address.png', 0)
    template_prev = cv2.imread('doc_prev_policy.png', 0)

    img = getBoxed(img.copy(), img_gray.copy(),
                        template_add, 'address')

    img = getBoxed(img.copy(), img_gray.copy(),
                   template_prev, 'prev_policy_no')

    cv2.imshow('Detected', img)
```

Smile detection

In [ ]:
```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +'haarcascade_frontalfac
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +'haarcascade_eye.xml')
smile_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +'haarcascade_smile.xml
```

In [ ]:
```
cap = cv2.VideoCapture(0)
```

In [ ]:
```
while True:
    ret, frame = cap.read()

    if not ret:
        print("Failed to grab frame.")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)

    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
        roi_gray = gray[y:y + h, x:x + w]

        smiles = smile_cascade.detectMultiScale(roi_gray, scaleFactor=1.8, minNeigh

        for (sx, sy, sw, sh) in smiles:
            cv2.rectangle(frame, (x + sx, y + sy), (x + sx + sw, y + sy + sh), (0,

    cv2.imshow('Smile Detection', frame)
```

In [ ]:
```
if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

Feature extraction and image classification using OpenCV

In [ ]:
```
pip install opencv-python
```

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [ ]:
```
image = cv2.imread('GeeksForGeeks.jpg')
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

In [ ]:
```
edges = cv2.Canny(gray_image, 100, 200)
plt.imshow(edges, cmap='gray')
plt.title('Edge Image')
plt.show()
```

In [ ]:
```
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray_image, None)
image_with_sift = cv2.drawKeypoints(image, keypoints, None)
plt.imshow(cv2.cvtColor(image_with_sift, cv2.COLOR_BGR2RGB))
plt.title('SIFT Features')
plt.show()
```

In [ ]:
```
orb = cv2.ORB_create()
keypoints, descriptors = orb.detectAndCompute(gray_image, None)
image_with_orb = cv2.drawKeypoints(image, keypoints, None, flags=cv2.DRAW_MATCHES_F
plt.imshow(cv2.cvtColor(image_with_orb, cv2.COLOR_BGR2RGB))
plt.title('ORB Features')
plt.show()
```

In [ ]:
```
pip install PyWaveletspip install opencv-python
```

In [ ]:
```
import numpy as np
import cv2
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
```

In [ ]:
```
img = cv2.imread('images\ms_dhoni\dhoni.jpg')
plt.imshow(img)
```

In [ ]:
```
#Converting the image to gray
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray.shape
plt.imshow(gray)
```

In [ ]:
```
plt.imshow(gray, cmap='gray')
```

In [ ]:
```
face_cascade = cv2.CascadeClassifier('./opencv/haarcascades/haarcascade_frontalface
eye_cascade = cv2.CascadeClassifier('./opencv/haarcascades/haarcascade_eye.xml')

faces = face_cascade.detectMultiScale(gray, 1.3, 5)
faces
```

In [ ]:
```
array([[172,  55, 268, 268]])
```

In [ ]:
```
(x,y,w,h) = faces[0]
x,y,w,h
```

In [ ]:
```
(172, 55, 268, 268)
```

In [ ]:
```
face_img = cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
plt.imshow(face_img)
```

In [ ]:
```
cv2.destroyAllWindows()
for (x,y,w,h) in faces:
    face_img = cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = face_img[y:y+h, x:x+w]
    eyes = eye_cascade.detectMultiScale(roi_gray)
    for (ex,ey,ew,eh) in eyes:
        cv2.rectangle(roi_color,(ex,ey),(ex+ew,ey+eh),(0,255,0),2)


plt.figure()
plt.imshow(face_img, cmap='gray')
plt.show()
```

In [ ]:
```
#Plotting the cropped Image
%matplotlib inline
plt.imshow(roi_color, cmap='gray')
plt.show()
```

In [ ]:
```
cropped_img = np.array(roi_color)
cropped_img.shape
```

In [ ]:
```
import numpy as np
import pywt
import cv2

def w2d(img, mode='haar', level=1):
    imArray = img
    #Datatype conversions
    #convert to grayscale
    imArray = cv2.cvtColor( imArray,cv2.COLOR_RGB2GRAY )
    #convert to float
    imArray =  np.float32(imArray)
    imArray /= 255;
    # compute coefficients
    coeffs=pywt.wavedec2(imArray, mode, level=level)

    #Process Coefficients
    coeffs_H=list(coeffs)
    coeffs_H[0] *= 0;

    # reconstruction
    imArray_H=pywt.waverec2(coeffs_H, mode);
    imArray_H *= 255;
    imArray_H =  np.uint8(imArray_H)
```

```
    return imArray_H
```

In [ ]:
```
im_har = w2d(cropped_img,'db1',5)
plt.imshow(im_har, cmap='gray')
```

In [ ]:
```
def get_cropped_image_if_2_eyes(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x,y,w,h) in faces:
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = img[y:y+h, x:x+w]
        eyes = eye_cascade.detectMultiScale(roi_gray)
        if len(eyes) >= 2:
            return roi_color

original_image = cv2.imread('images\ms_dhoni\dhoni.jpg')
plt.imshow(original_image)
```

In [ ]:
```
cropped_image = get_cropped_image_if_2_eyes('images\ms_dhoni\dhoni.jpg')
plt.imshow(cropped_image)
```

In [ ]:
```
path_to_data = "./images/"
path_to_cr_data = "./images/cropped/"

import os
img_dirs = []
for entry in os.scandir(path_to_data):
    if entry.is_dir():
        img_dirs.append(entry.path)
img_dirs
```

In [ ]:
```
['./images/bhuvneshwar_kumar', './images/dinesh_karthik', './images/hardik_pandya',
```

In [ ]:
```
import shutil
if os.path.exists(path_to_cr_data):
     shutil.rmtree(path_to_cr_data)
os.mkdir(path_to_cr_data)
```

In [ ]:
```
cropped_image_dirs = []
celebrity_file_names_dict = {}

for img_dir in img_dirs:
    count = 1
    celebrity_name = img_dir.split('/')[-1]
    print(celebrity_name)

    celebrity_file_names_dict[celebrity_name] = []

    for entry in os.scandir(img_dir):
        roi_color = get_cropped_image_if_2_eyes(entry.path)
        if roi_color is not None:
            cropped_folder = path_to_cr_data + celebrity_name
```

```
            if not os.path.exists(cropped_folder):
                os.makedirs(cropped_folder)
                cropped_image_dirs.append(cropped_folder)
                print("Generating cropped images in folder: ",cropped_folder)

            cropped_file_name = celebrity_name + str(count) + ".png"
            cropped_file_path = cropped_folder + "/" + cropped_file_name

            cv2.imwrite(cropped_file_path, roi_color)
            celebrity_file_names_dict[celebrity_name].append(cropped_file_path)
            count += 1
```

In [ ]:
```
bhuvneshwar_kumarGenerating cropped images in folder:  ./images/cropped/bhuvneshwar
```

In [ ]:
```
celebrity_file_names_dict = {}
for img_dir in cropped_image_dirs:
    celebrity_name = img_dir.split('/')[-1]
    file_list = []
    for entry in os.scandir(img_dir):
        file_list.append(entry.path)
    celebrity_file_names_dict[celebrity_name] = file_list
celebrity_file_names_dict
```

In [ ]:
```
{'bhuvneshwar_kumar': ['./images/cropped/bhuvneshwar_kumar\\bhuvneshwar_kumar1.png'
```

In [ ]:
```
class_dict = {}
count = 0
for celebrity_name in celebrity_file_names_dict.keys():
    class_dict[celebrity_name] = count
    count = count + 1
class_dict
```

In [ ]:
```
{'bhuvneshwar_kumar': 0, 'dinesh_karthik': 1, 'hardik_pandya': 2, 'jasprit_bumrah':
```

In [ ]:
```
X, y = [], []
for celebrity_name, training_files in celebrity_file_names_dict.items():
    for training_image in training_files:
        img = cv2.imread(training_image)
        if img is None:
            continue
        scalled_raw_img = cv2.resize(img, (32, 32))
        img_har = w2d(img,'db1',5)
        scalled_img_har = cv2.resize(img_har, (32, 32))
        combined_img = np.vstack((scalled_raw_img.reshape(32*32*3,1),scalled_img_ha
        X.append(combined_img)
        y.append(class_dict[celebrity_name])
```

In [ ]:
```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

```
pipe = Pipeline([('scaler', StandardScaler()), ('svc', SVC(kernel = 'rbf', C = 10))
pipe.fit(X_train, y_train)
pipe.score(X_test, y_test)
```

In [ ]:
```
0.5166666666666667
```

In [ ]:
```
print(classification_report(y_test, pipe.predict(X_test)))
```

In [ ]:
```
precision    recall  f1-score   support           0       0.50      0.25      0.33
```

Feature Detection & Analysis

Find Co-ordinates of Contours

In [ ]:
```
import numpy as np
import cv2

font = cv2.FONT_HERSHEY_COMPLEX

# Load image
img2 = cv2.imread('test.jpg', cv2.IMREAD_COLOR)
img = cv2.imread('test.jpg', cv2.IMREAD_GRAYSCALE)

# Binarize image
_, threshold = cv2.threshold(img, 110, 255, cv2.THRESH_BINARY)

# Find contours
contours, _ = cv2.findContours(threshold, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    # Approximate and draw contour
    approx = cv2.approxPolyDP(cnt, 0.009 * cv2.arcLength(cnt, True), True)
    cv2.drawContours(img2, [approx], 0, (0, 0, 255), 5)

    # Flatten points
    n = approx.ravel()
    i = 0
    for j in n:
        if i % 2 == 0:  # x, y coords
            x, y = n[i], n[i + 1]
            coord = f"{x} {y}"
            if i == 0:  # first point
                cv2.putText(img2, "Arrow tip", (x, y), font, 0.5, (255, 0, 0))
            else:
                cv2.putText(img2, coord, (x, y), font, 0.5, (0, 255, 0))
        i += 1

# Show result
cv2.imshow('Contours with Coordinates', img2)

# Exit on 'q'
```

```
if cv2.waitKey(0) & 0xFF == ord('q'):
    cv2.destroyAllWindows()
```

Analyze an image using Histogram

In [ ]:
```
import matplotlib.pyplot as plt
import cv2

img = plt.imread('flower.png')
plt.imshow(img)
plt.title("Original Image")
plt.show()
```

In [ ]:
```
import cv2, matplotlib.pyplot as plt
img = cv2.imread("flower.png")

# Grayscale + Histogram
g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.subplot(121), plt.imshow(g, cmap='gray'), plt.axis("off"), plt.title("Grayscale
plt.subplot(122), plt.hist(g.ravel(),256,[0,256],color='k'), plt.title("Gray Histog
plt.show()

# RGB Histograms
for i,c in enumerate(('r','g','b')):
    plt.plot(cv2.calcHist([img],[i],None,[256],[0,256]), color=c)
plt.title("RGB Histograms"), plt.xlabel("Intensity"), plt.ylabel("Frequency")
plt.show()
```

In [ ]:
```
img = cv2.imread('flower.jpg', 0)
histg = cv2.calcHist([img], [0], None, [256], [0, 256])

# Plot histogram
plt.plot(histg)
plt.title("Histogram using OpenCV calcHist()")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
```

In [ ]:
```
img = cv2.imread('ex.jpg')

# colors for channels
colors = ('b', 'g', 'r')

for i, col in enumerate(colors):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=col)
    plt.xlim([0, 256])

plt.title("RGB Color Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
```

In [ ]:
```
import cv2
from matplotlib import pyplot as plt
img = cv2.imread('ex.jpg',0)

histr = cv2.calcHist([img],[0],None,[256],[0,256])
plt.plot(histr)
plt.show()
```

In [ ]:
```
import cv2
from matplotlib import pyplot as plt
img = cv2.imread('ex.jpg',0)

# alternative way to find histogram of an image
plt.hist(img.ravel(),256,[0,256])
plt.show()
```